

Automatizar testes de acessibilidade em Java é totalmente viável — e hoje já existem ferramentas bem maduras para isso. O ponto importante: testes automatizados **não substituem validação manual**, mas ajudam muito a detectar problemas básicos rapidamente (especialmente alinhados com WCAG).

1. Usando Axe (recomendado)

A biblioteca mais usada é o **axe-core**, da Deque Systems, integrada com Selenium.



Dependência (Maven)

```
<dependency>

<groupId>com.deque.html.axe-core</groupId>

<artifactId>selenium</artifactId>

<version>4.7.2</version>

</dependency>
```



Exemplo com Selenium + Axe

```
import com.deque.html.axecore.selenium.AxeBuilder;

import com.deque.html.axecore.results.AxeResults;

import org.junit.jupiter.api.Test;

import org.openqa.selenium.WebDriver;

import org.openqa.selenium.chrome.ChromeDriver;


public class AcessibilidadeTest {

    @Test

    public void deveValidarAcessibilidade() {

        WebDriver driver = new ChromeDriver();

        driver.get("https://example.com");


        AxeResults results = new AxeBuilder().analyze(driver);
```

```

    if (!results.getViolations().isEmpty()) {
        results.getViolations().forEach(v -> {
            System.out.println("Impacto: " + v.getImpact());
            System.out.println("Descrição: " + v.getDescription());
            System.out.println("Ajuda: " + v.getHelpUrl());
            System.out.println("-----");
        });
    }

    driver.quit();
}
}

```

O que esse teste valida?

- Imagens sem alt
- Contraste de cores
- Labels ausentes
- Estrutura semântica
- Problemas de ARIA

2. Integrando com Playwright (Java)

Se você estiver usando Playwright, também dá pra integrar Axe:

```

import com.microsoft.playwright.*;

import com.deque.html.axecore.playwright.*;

public class AxePlaywrightTest {

    public static void main(String[] args) {

        try (Playwright playwright = Playwright.create()) {

            Browser browser = playwright.chromium().launch();

```

```
Page page = browser.newPage();

page.navigate("https://example.com");

AxePlaywright axe = new AxePlaywright(page);
AxeResults results = axe.analyze();

System.out.println(results.getViolations());

browser.close();
}
}
}
```

3. Criando um teste mais realista (login)

```
@Test
public void testeAcessibilidadeLogin() {
    WebDriver driver = new ChromeDriver();
    driver.get("http://localhost:8080/login.html");

    AxeResults results = new AxeBuilder()
        .include("#formLogin") // valida só o form
        .analyze(driver);

    assertFalse(results.getViolations().isEmpty(),
        "Foram encontradas violações de acessibilidade!");

    driver.quit();
}
```

4. Gerando relatório

Você pode salvar o resultado em JSON:

```
FileWriter writer = new FileWriter("relatorio-acessibilidade.json");  
  
writer.write(results.toString());  
  
writer.close();
```

5. Ferramentas complementares

Além do Axe:

- Lighthouse (Chrome DevTools)
- Pa11y
- Wave

6. Evolução do projeto (nível profissional)

Você pode evoluir para:

✔ Validar páginas específicas

```
new AxeBuilder()  
  
    .include("#formLogin")  
  
    .analyze(driver);
```

✔ Falhar só erros críticos

```
results.getViolations().stream()  
  
    .filter(v -> "critical".equals(v.getImpact()))
```

7. Em que o Axe se baseia?

✔ 1. WCAG

É a principal referência.

As WCAG são organizadas em 4 princípios:

◆ 1. Perceptível

O conteúdo deve ser visível/audível para todos

Ex:

- Imagens precisam de alt
- Contraste adequado

◆ 2. Operável

Usuário consegue navegar/interagir

Ex:

- Navegação por teclado
- Botões acessíveis

◆ 3. Compreensível

Interface previsível

Ex:

- Labels claros
- Mensagens de erro

◆ 4. Robusto

Compatível com tecnologias assistivas

Ex:

- HTML semântico
- Uso correto de ARIA

2. Regras internas do Axe

O Axe implementa essas diretrizes como **regras automatizadas**.

Cada violação tem:

- id → regra (ex: image-alt)
- impact → gravidade (minor, moderate, serious, critical)
- description → explicação

- `helpUrl` → documentação

Exemplo real de regra

Regra: `image-alt`

 Baseada na WCAG 1.1.1

✓ Exige:

```

```

✗ Erro:

```

```

3. Outras bases além da WCAG

O Axe também considera:

✓ **WAI-ARIA**

- uso correto de `aria-label`, `role`, etc.

✓ **Boas práticas de HTML**

- uso correto de `<label>`
- estrutura semântica (`<header>`, `<nav>`, etc.)

✓ **Padrões de usabilidade**

- foco visível
- hierarquia de headings

Limitação importante (muito importante mesmo)

O Axe cobre **aproximadamente 30% a 40% dos problemas reais**.

 Ele **NÃO** consegue validar:

- Se o `alt` faz sentido (semântica)

- Se o texto é claro
- Experiência com leitor de tela
- Fluxo de navegação real

Exemplo clássico

✓ Isso PASSA no Axe:

```

```

✗ Mas é ruim na prática(o alt não descreve nada útil)